



FoodDraft

Documentación técnica y funcional de la aplicación

App web (PWA) para generar menús semanales variados a partir de tus propias comidas, sin recomendaciones externas, con control de frecuencias por alimento, cálculo automático de objetivos nutricionales, y un plan premium con seguimiento de peso, lector de etiquetas por foto y sugerencias de cocina con IA. Disponible en español, inglés, catalán, gallego y euskera.

1. Qué es la aplicación y por qué existe

FoodDraft es una aplicación web (PWA — Progressive Web App), pensada para resolver un problema muy concreto: dejar de tener que pensar qué comer en cada comida, sin que ninguna app decida por ti *qué* comer. La idea de fondo es que tú aportas tus propios platos íntegros (los que ya sueles cocinar), y la aplicación se limita a combinarlos en un menú semanal variado, respetando reglas de frecuencia que tú mismo defines.

Nació como app de uso estrictamente personal; desde septiembre de 2026 está en proceso de abrirse a más usuarios (registro por email o Google, ya sin restricción a nadie), con una capa de suscripción premium para las funciones con coste de servidor o valor añadido claro (ver secciones 7 y 8). La interfaz está disponible en español, inglés, catalán, gallego y euskera.

Los dos objetivos que guían todo el diseño son:

- **Variar** las comidas semana a semana sin esfuerzo mental, evitando caer siempre en lo mismo.
- **Evitar atracones** — dar cierta estructura y previsibilidad a la alimentación, sin llegar a un control obsesivo de cada gramo, para tener una relación más tranquila con la comida.

Un principio importante que se ha mantenido en todo el desarrollo: la app no elige qué es un plato ni lo desglosa por ti — tú decides que "Pollo + Arroz + Ensalada" es una comida válida, con qué frecuencia quieres que aparezca, y la app solo organiza la variedad y las cantidades dentro de esa estructura que ya has definido.

2. Arquitectura técnica

La app está montada sin proceso de compilación (build): es HTML + JSX que el propio navegador transforma al vuelo con Babel Standalone, cargado por *import maps* directamente desde CDN (esm.sh). Esto la hace muy fácil de editar y desplegar (basta con subir los archivos a GitHub Pages), a cambio de una carga inicial algo más lenta que con un build optimizado. Esta decisión se mantiene sin cambios desde el inicio — ver CLAUDE.md, "Regla más importante: no hay build".

Archivo / carpeta	Función
<code>index.html</code>	Punto de entrada: carga el import map, Babel, registra el service worker y arranca <code>auth-bootstrap.jsx</code> .
<code>auth-bootstrap.jsx</code>	Login con Firebase Auth (email/contraseña y Google) y App Check. Expone en <code>window.*</code> toda la interacción con Firebase (storage, premium, Stripe, IA, notificaciones push) para que <code>app.jsx</code> no dependa de Firebase directamente. Una vez autenticado, monta <code>app.jsx</code> .
<code>app.jsx</code>	Todos los componentes React + el componente principal: pantallas, catálogo de alimentos e ingredientes, perfil, premium. Importa la capa de lógica de abajo (~7.200 líneas, deliberadamente sin dividir — ver CLAUDE.md).
<code>Logica/*.js</code>	Capa de lógica sin JSX ni React: <code>comun.js</code> , <code>macros.js</code> , <code>seleccion.js</code> , <code>comida-calculo.js</code> , <code>objetivos.js</code> (TDEE, etapas, macros, presets de reparto de comidas), <code>menu-generador.js</code> (motor de generación), <code>peso.js</code> (seguimiento de peso), <code>salud-publica.js</code> , <code>resumenMensual.js</code> e <code>i18n.js</code> (textos de la interfaz en cinco idiomas).
<code>functions/*.js</code>	Cloud Functions (Firebase, 2ª gen, Node 22) — la única pieza de servidor propio del proyecto: pagos, IA con cuota y recordatorio push. Es un proyecto Node aparte, con su propio <code>package.json</code> . Ver secciones 7 y 8.

Archivo / carpeta	Función
<code>manifest.json</code>	Metadatos de PWA: nombre ("FoodDraft"), icono, colores — permite "Añadir a pantalla de inicio".
<code>sw.js</code>	Service worker: cachea los archivos con estrategia "red primero" para que funcione offline y se actualice solo al haber conexión, y muestra las notificaciones push cuando la app está cerrada.
<code>privacidad.html</code> / <code>terminos.html</code>	Política de privacidad y términos de uso, enlazados desde el login. Necesarios para publicar en Google Play (ver hoja de ruta de publicación).
<code>icon.svg</code>	Icono de la app.

Guardado de datos: Firebase Firestore, ligado a tu cuenta de usuario. La app de datos (ingredientes, frecuencias, perfil, historial de menús) vive en `users/{uid}/keys/{key}`, de lectura/escritura libre para el propio usuario. Aparte, el documento `users/{uid}` guarda el estado `premium` y los contadores de cuota (`usoFotos`, `usoCocinar`) — de **solo lectura para el cliente**: únicamente las Cloud Functions, con el Admin SDK, pueden escribir ahí. Esta separación es la base de todo el sistema de premium (sección 7).

Despliegue: la app se sirve desde GitHub Pages, sin paso de build: un push a la rama principal llega a la app real en cuestión de minutos, sin ningún despliegue manual (y sin CI que lo frene, así que conviene pasar por `Validacion/` antes de subir nada). Las Cloud Functions, en cambio, se despliegan a mano con `firebase deploy --only functions`.

3. Cómo se organizan los ingredientes

Cada ingrediente que das de alta pertenece a una categoría y tiene un tipo de regla que decide cómo de a menudo aparece en el menú. Esta combinación es el corazón de todo el sistema de variedad.

3.1 Categorías

Categoría	Qué es	Ejemplos del catálogo actual
proteína	El componente principal de comida/cena.	Pollo, salmón, merluza, atún, sardinas, carne picada, hamburguesas
carbo	La base de hidratos de la comida.	Pasta, arroz, gnocchi, patatas
verdura	El acompañamiento vegetal.	Ensalada completa, ensalada de tomate, pimiento y cebolla, purés
grasa	Grasas de ajuste (aceite, frutos secos, guacamole...).	Aceite de oliva, pistachos, anacardos, guacamole
desayuno / media mañana / merienda	Combinaciones fijas para esas franjas (la media mañana solo existe en el reparto de 5 comidas).	Definidas por el usuario
especial	Modificadores que se añaden sobre una comida ya elegida.	Garbanzos (se suma a la ración de carbo)

Categoría	Qué es	Ejemplos del catálogo actual
cerrado	Platos ya completos, cerrados, que sustituyen a la estructura proteína+carbo+verdura.	Hamburguesa completa con pan, nachos, huevos rotos

3.2 Tipos de regla (cómo se decide la frecuencia)

Tipo de regla	Cómo funciona
base	El "relleno por defecto": aparece en todos los huecos que no ocupa ninguna otra regla. Es el caso de Pollo (carne) — es intencionadamente el ingrediente más repetido, en torno a la mitad de las comidas/cenas del ciclo.
frecuencia	Aparece un número fijo de veces por período (semana o ciclo completo de 2 semanas). Es el mecanismo pensado para "esto no puede faltar, pero tampoco saturar" — por ejemplo el pescado.
probabilidad	Compite por un peso relativo (%) frente a otros ingredientes de su misma categoría. Se usa para elementos intercambiables entre sí, como los carbohidratos base (pasta/arroz/gnocchi/patatas) o las verduras.
bloque_miembro	Varios ingredientes agrupados en un "bloque": el bloque tiene su propia frecuencia total, y dentro se reparte entre sus miembros según su peso relativo. Así, "carne roja" reparte entre picada de cerdo/vacuno y hamburguesa de ternera/cerdo sin que compitan con el resto del catálogo.

Las frecuencias concretas configuradas hoy (proteínas, garbanzos, probabilidades de carbohidrato y verdura, platos cerrados) están detalladas aparte en «Frecuencias y probabilidades», junto con las reglas de afinidad activas.

4. El motor de generación del menú

Cuando pulsas "Generar menú", la app construye un ciclo de 2 semanas: 14 días × las comidas de tu reparto (de 2 a 5 al día, ver 6.4). Comida y cena —28 huecos por ciclo— reciben el sorteo completo de proteína, carbohidrato y verdura; desayuno, media mañana y merienda se reparten aparte. El proceso es este:

- **Reparto de huecos por frecuencia:** primero se calcula cuántas veces debe aparecer cada ingrediente de tipo "frecuencia" o "bloque" en el ciclo, y se reservan esos huecos concretos (método del resto mayor, para repartir de forma justa incluso cuando los números no encajan exactos).
- **Relleno con la base:** los huecos que sobran se rellenan con el ingrediente "base" (Pollo).
- **Elección de carbohidrato y verdura** para cada comida, por sorteo ponderado según las probabilidades que hayas configurado.
- **Reglas de combinación (afinidad):** puedes definir reglas del tipo "si sale Salmón, sube la probabilidad de que la verdura sea Ensalada completa" (niveles: nula/baja/alta/máxima). Estas reglas solo se evalúan cuando el resto de elementos de esa combinación ya están decididos.
- **Desayuno, media mañana y merienda:** reparto exacto según las probabilidades, no un sorteo independiente por comida — con solo 14 huecos de cada tipo, un sorteo puro se aleja demasiado del porcentaje real (como lanzar una moneda 14 veces no siempre da 7 y 7). Se calcula cuántas veces le toca a cada opción (resto mayor) y después se baraja el orden.

- **Anti-repetición:** el generador recuerda la última elección de cada grupo de un solo hueco (por historial) para evitar repetir exactamente lo mismo en el ciclo siguiente cuando hay alternativa disponible. *Dentro* de un mismo ciclo, en cambio, carbo y verdura se sortean de forma independiente en cada comida, así que puede haber rachas del mismo alimento — está anotado como mejora pendiente (`Ideas futuras.md`).
- **Ajuste de raciones a tus objetivos:** si tienes un perfil con objetivos calculados, la proteína y el carbohidrato de cada comida se ajustan para acercarse a su parte proporcional del objetivo; la grasa queda como resultado, con el aceite de oliva (o una grasa de tu pestaña "Grasas") como "mando de ajuste fino" visible. Los platos cerrados y las comidas de una pieza (desayuno, media mañana, merienda) escalan su receta completa de forma proporcional para acercarse a las kcal de esa comida (clamp 0,3×–4×), preservando las proporciones internas de sus ingredientes.

Es importante remarcar la filosofía detrás de este último punto: no existe una única combinación de gramos que cuadre exactamente los 4 objetivos (kcal, proteína, grasa, carbohidrato) cada día — matemáticamente hay infinitas soluciones posibles, y algunos días el menú no tiene ningún alimento con mucha grasa, así que no se puede inventar de la nada. El objetivo realista es acercarse mucho, con la **media semanal** como referencia más fiable que un solo día.

4.1 Menú completo y Menú simple

La app ofrece dos pantallas de menú que usan exactamente el mismo motor. El **Menú completo** muestra kcal, macros y raciones ajustables, y compara cada día con tus objetivos. El **Menú simple** muestra solo el borrador de combinaciones, sin calorías ni gramos — pensado para quien prefiere no ver números. Cada una tiene su propio ciclo, su historial, su botón de deshacer y su lista de la compra: generar o deshacer en una nunca toca a la otra.

5. Base de datos de alimentos y cálculo de macros

Cada ingrediente del menú está enlazado a un alimento de un catálogo interno con sus macros por 100g (actualmente unos 43 alimentos, agrupados en carpetas: proteínas, embutido y fiambre, carbohidratos, panes, cereales, legumbres, vegetales, frutas, lácteos, grasas, frutos secos y snacks procesados; con calorías, proteína, grasa, carbohidratos, y en muchos casos también sal, azúcares, fibra y grasa saturada). El origen del dato queda marcado como "Etiqueta" (sacado del envase real del producto que usas) o "Estándar" (valor de referencia contrastado, para alimentos genéricos sin marca fija, como el tomate o la patata) — el detalle completo de valores está en «Valores nutricionales».

Con esto, cada comida generada calcula sus macros exactos escalando por la cantidad real de gramos, y esos totales se agregan por comida, por día y por semana — es lo que alimenta las gráficas de calorías y la "rueda" de macros de la app, y lo que hace posible comparar el menú generado contra tus objetivos.

6. Perfil y cálculo de objetivos nutricionales

La primera vez que entras, la app te guía con un asistente de perfil de 5 pasos (datos personales, estilo de vida, entrenamientos, objetivo y reparto de comidas) y, a continuación, un cuestionario de catálogo (exclusiones por gusto, proteínas, carbohidratos, verduras, grasas y las comidas ligeras de tu reparto) que construye tus ingredientes iniciales — las cuentas nuevas ya no heredan ningún catálogo ajeno. Ambos pasos se pueden saltar, y el cuestionario se puede repetir desde Ajustes. Después, estos datos solo se tocan desde Perfil y Actividad diaria, y cada vez que los guardas, los objetivos se recalculan automáticamente. El detalle completo de fórmulas, tablas y evidencia científica vive en «Fundamentos nutricionales y deportivos» — esta sección resume solo lo que hace falta para tocar código.

6.1 TDEE: PAL de base + METs de entrenamiento (`calcularObjetivosPerfil`)

El gasto energético total se calcula en dos piezas separadas, no con un único selector de actividad — esto sustituyó definitivamente al sistema antiguo de 4 niveles fijos (Sedentario/Ligero/Moderado/Alto con un factor único cada uno):

- **BMR** (Mifflin-St Jeor): $10 \times \text{peso} + 6,25 \times \text{altura} - 5 \times \text{edad} + 5 \text{ (H)} / -161 \text{ (M)}$
- **PAL de día a día** (excluye el entrenamiento): 1,25 / 1,45 / 1,65 según el tipo de día a día que indique el perfil.
- **Kcal de entrenamiento**, aparte, por cada entrenamiento habitual: $\text{MET} \times \text{peso} \times \text{horas/sesión} \times \text{sesiones/semana} \div 7$, sumadas todas.
- **TDEE de mantenimiento** = (BMR × PAL de día a día) + Kcal de entrenamiento.

6.2 Etapas: mantenimiento, volumen limpio, definición conservadora

Sobre el TDEE de mantenimiento, la persona elige una etapa con un ajuste porcentual fijo — nunca un recorte de kcal absoluto:

Etapa	Ajuste	Proteína
Mantenimiento	0%	según tabla por volumen de entrenamiento
Volumen limpio	+12%	fijo 1,6 g/kg
Definición conservadora	-15%	fijo 2,2 g/kg

Techo de seguridad asimétrico: el ajuste (etapa + calibración por seguimiento de peso, si la hay) nunca puede superar -17% en déficit ni +15% en superávit — un ajuste más suave de lo calculado no tiene techo, porque no supone riesgo.

6.3 Macros: proteína, grasa con suelo hormonal, carbohidrato con suelo deportivo

Proteína y grasa salen de una tabla por sesiones de entrenamiento/semana (0 / 1-2 / 3-5 / 6-7): proteína 1,0 / 1,3 / 1,6 / 1,6 g/kg y grasa 0,85 / 0,90 / 1,00 / 1,25 g/kg — salvo en volumen/definición, donde la proteína pasa a su valor fijo de etapa (1,6 y 2,2 g/kg). La grasa nunca baja de $\text{máx}(0,6 \text{ g/kg} ; 20\% \text{ de las kcal totales})$, y ese suelo se aplica siempre, no solo cuando otro ajuste la empuja hasta él. El carbohidrato ocupa lo que sobra de kcal, salvo que haya entrenamiento habitual, en cuyo caso tiene su propio suelo:

Situación	Suelo de carbohidrato	Cómo se mide
Fuera de definición · carga baja	2,5 g/kg	< 1,5 kcal de entrenamiento por kg de peso y día
Fuera de definición · carga media	3,5 g/kg	1,5 – 3,5 kcal/kg/día
Fuera de definición · carga alta	4,0 g/kg	> 3,5 kcal/kg/día
En definición (déficit)	2,6 g/kg, plano	no escala con el entrenamiento

Si el carbohidrato calculado no llega a su suelo, se le ceden kcal a la grasa hasta donde alcance su propio suelo — nunca a costa de la proteína. Si aun así no llega, se acepta que se quede corto y la app muestra un aviso explícito. El porqué de cada

cifra (y por qué en déficit el suelo es distinto) está en «Fundamentos nutricionales y deportivos», sección 4, y en «Evidencia científica», secciones 4 y 5.

6.4 Reparto por comida

El objetivo diario se reparte entre las comidas del día según el reparto de cada perfil (`perfil.repartoComidas` , de 2 a 5 comidas; Comida y Cena siempre están). Hay cinco patrones a elegir en Configuración de comidas → Reparto de comidas: **Clásico · 4 comidas** (Desayuno 20% · Comida 35% · Merienda 15% · Cena 30%, el de partida y el marcado como recomendado), **3 comidas sin merienda**, **3 comidas sin desayuno**, **5 comidas con media mañana** y **2 comidas** (comida y cena, ayuno intermitente). Los porcentajes los pone la persona libremente, con la única condición de que sumen 100% (`objetivosPorComida`). Cada patrón lleva un texto educativo con la evidencia disponible, y la Cena un aviso fijo sobre cenas copiosas o muy tardías. El número de comidas al día que usa el resumen mensual se deriva de este reparto.

Aviso que se mantiene: ni este método afinado deja de ser una estimación (margen de error habitual $\pm 10\text{-}15\%$). El paso final de calibración sigue siendo pesarse periódicamente comiendo al objetivo calculado y ajustar según la tendencia real del peso — eso corrige cualquier fórmula mejor que cualquier fórmula por sí sola. Este paso de calibración vive hoy dentro del seguimiento de peso, que es una función premium (sección 8).

7. Premium y pagos

Añadido en septiembre de 2026 — la primera pieza de servidor propio del proyecto. Antes de esto, todo era cliente + Firestore, sin backend.

7.1 Cómo se cobra

Con Stripe, gestionado en la propia web — no con Google Play Billing. 2,99€/mes. El flujo:

1. El cliente llama a la Cloud Function `createCheckoutSession` , que crea una sesión de Stripe Checkout con `client_reference_id: uid` y `subscription_data.metadata.firebaseUID: uid` , para poder mapear el evento de vuelta al usuario de Firebase.
2. Stripe redirige al pago; al completarse, envía un evento al endpoint `stripeWebhook` (verificado con la cabecera `stripe-signature` , no con App Check — es Stripe quien llama, no el cliente de la app).
3. El webhook escribe `premium.active = true` (y guarda `stripeCustomerId`) en `users/{uid}` , usando el Admin SDK.
4. El cliente se entera en vivo por `window.subscribePremiumStatus` (listener de Firestore en `auth-bootstrap.jsx`), sin recargar la página.
5. Cancelar o cambiar de tarjeta pasa por `createPortalSession` , que abre el portal de facturación de Stripe usando el `stripeCustomerId` guardado.

Por qué el campo `premium` es de solo lectura para el cliente: cualquiera con la consola del navegador abierta podría, si no fuera así, escribirse a mano ese campo en su propio documento. Las reglas de Firestore de `users/{uid}` solo permiten lectura al propio usuario; la escritura únicamente la hace el Admin SDK desde las Cloud Functions.

7.2 Control de coste de las funciones con IA

Un límite "solo de cara al usuario" en la interfaz no protegería nada de verdad: si la llamada a Gemini saliera directa desde el navegador, cualquiera podría saltarse el límite llamando a la función a mano desde la consola. Por eso, cada función que llama a Gemini (lector de fotos, "qué puedo cocinar") reserva su hueco de cuota **dentro de una transacción de Firestore, antes de llamar a la IA** — comprobando `premium.active` y el contador del mes — y solo si la reserva tiene éxito se hace la llamada. Si la llamada a Gemini falla después, el hueco se devuelve (`FieldValue.increment(-1)` , no transaccional, pero solo hace el cupo más generoso, nunca lo rompe).

Cloud Function	Cuota	Campo del contador
<code>analyzeFoodPhoto</code>	20 al mes	<code>usoFotos</code>
<code>suggestMeals</code>	10 al mes	<code>usoCocinar</code>

8. Funciones premium

Gratis para siempre: el generador de menús (completo y simple), el cálculo de objetivos, el catálogo de alimentos y la lista de la compra (secciones 3 a 6). De pago, porque tienen coste real de servidor o aportan valor claramente extra:

8.1 Seguimiento de peso (`Logica/peso.js`)

Ciclos de pesadas (2-4 semanas) que se ocultan hasta el cierre, detección de cambios atípicos ($\geq 1,5\%$ de la última pesada normal), cálculo de tendencia por regresión lineal, sugerencia de ajuste de calorías, pausas del ciclo (viajes, enfermedad), historial permanente y exportación a PDF. Diseño completo y evidencia científica en «Fundamentos nutricionales y deportivos», sección 9.

8.2 Lector de etiquetas por foto (`functions/analyzeFoodPhoto.js`)

Foto de una tabla nutricional → Gemini rellena el alimento. El nombre del modelo (`gemini-3.1-flash-lite`) está hardcodeado en esta función y en `suggestMeals.js` ; si Google lo retira, hay que cambiarlo en los dos archivos.

8.3 "¿Qué puedo cocinar con lo que tengo?" (`functions/suggestMeals.js`)

Foto de la nevera/despensa, texto escrito a mano, o ambas cosas. A diferencia del lector de fotos, aquí no se deja que Gemini invente ingredientes ni macros: se le manda el catálogo de alimentos del propio usuario (id + nombre) y se le pide que identifique cuáles de esos alimentos concretos reconoce — el servidor filtra cualquier `foodId` devuelto que no exista de verdad en el catálogo enviado. Las combinaciones sugeridas usan la misma forma `composicion: [{foodId, gramos}]` que un plato cerrado, así que se guardan directamente con el `EditModal` ya existente, con macros calculadas por `composedMacros` , nunca por la IA.

8.4 Resumen mensual (`Logica/resumenMensual.js` — disponible en ambos planes)

Cada comida del menú tiene un botón para marcarla como completada el día real en que se come (`data.comidasCompletadas` , por fecha real, independiente de la estructura de semanas del menú). En el plan gratis se ve la fracción de comidas completadas del mes, sobre el número de comidas de tu reparto (2 a 5, ver 6.4) × días transcurridos. En premium se añade el objetivo actual y la variación de peso del mes con su gráfica, reutilizando el cálculo de tendencia de `peso.js` .

8.5 Sin anuncios

No hay ningún anuncio implementado todavía en ningún plan. Se promete en la pantalla premium porque es cierto por defecto; implementar anuncios de verdad en el plan gratis es un proyecto aparte, previsto antes de publicar en la tienda (no después, para que no se perciba como algo añadido a una app que la gente ya conocía sin ellos).

8.6 Notificaciones push: recordatorio de pesaje

Desde Ajustes se puede activar un recordatorio: la app pide permiso al navegador, obtiene un token de Firebase Cloud Messaging y lo guarda en `users/{uid}/keys/rueda-de-platos:push-token-v1`. Una Cloud Function programada (`sendWeighInReminders` , todos los días a las 9:00 hora de España) avisa a quien tenga hoy día de pesaje sugerido, no se haya pesado ya y no tenga el ciclo en pausa; el mensaje sale en el idioma del perfil. `sw.js` muestra la notificación con la app cerrada. En iPhone (Safari) solo llega si la PWA está instalada de verdad — no se puede evitar con código. Con el volumen de uso actual no supone ningún coste.

9. Eliminación de cuenta y RGPD

Botón "Eliminar cuenta" en Ajustes, con reautenticación obligatoria y confirmación escrita — borra Auth y todo el árbol de Firestore del usuario (`users/{uid}` y `users/{uid}/keys/*`). Es un requisito de Google Play desde 2023 para cualquier app que permita crear cuentas, no algo opcional. La política de privacidad (`privacidad.html`) recoge los derechos RGPD (acceso, rectificación, supresión, oposición, portabilidad, limitación).

10. Notas para retomar el desarrollo

- Antes de dar cualquier cambio en la lógica por terminado, pasa por `Validacion/` (debe decir "X / X casos superados") — ver CLAUDE.md, sección "Cómo verificar un cambio antes de darlo por bueno".
- Al añadir un módulo nuevo en `Logica/` , regístralo en los dos `importmap` (`index.html` y `Validacion/index.html`) y en `FILES_TO_CACHE` de `sw.js` , subiendo en uno `CACHE_NAME` .
- Cualquier cambio en la forma de los datos guardados necesita su parche correspondiente en `migrateData` (dentro de `app.jsx`) — nunca asumas que los datos ya guardados tienen la forma nueva.
- Todo texto de interfaz nuevo va como clave en los cinco diccionarios de `Logica/i18n.js` , no suelto en el JSX. Los datos del usuario y los identificadores del modelo de datos (nombres de comida, días) nunca se traducen.
- `sendWeighInReminders.js` repite a mano `esDiaSugeridoPeso` de `Logica/peso.js` (un módulo de navegador no se puede importar desde Node); si cambia la regla de días sugeridos, hay que cambiarla en los dos sitios.
- El nombre de cara al usuario es "FoodDraft"; el proyecto de Firebase, el repositorio de GitHub y las claves de Firestore siguen llamándose `rueda-de-platos` a propósito — no los renombres.

Fin de la documentación. Generado a partir del código fuente actual de la app (`app.jsx` , `auth-bootstrap.jsx` , `index.html` , `manifest.json` , `sw.js` , `Logica/` , `functions/`) y del histórico de decisiones tomadas en las conversaciones previas de diseño.